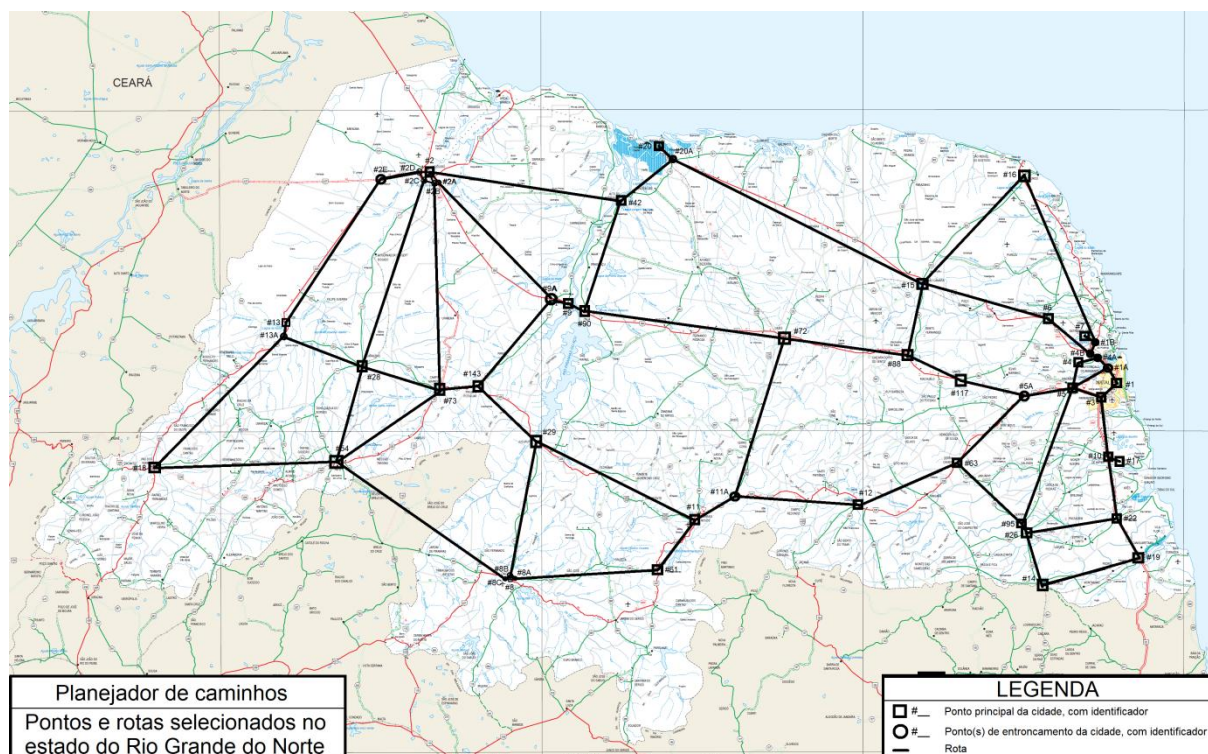


PLANEJADOR DE CAMINHOS
PROFESSOR: ADELARDO ADELINO DANTAS DE MEDEIROS



O objetivo é utilizar exceções, leitura de arquivos e a biblioteca STL de C++ para determinar o caminho mais curto entre dois pontos (origem e destino) em um mapa, usando o algoritmo A*. O mapa contém uma série de pontos, com conexões entre alguns deles (rotas). Cada rota tem um comprimento associado.

CAMINHO DE MENOR CUSTO EM GRAFO

O algoritmo A* encontra o caminho de menor custo em um grafo no qual a transição entre nós conectados tem um custo associado. O A* mantém um conjunto dos nós já visitados (**Fechado**) e um conjunto dos nós ainda não analisados (**Aberto**). No início, **Fechado** está vazio e **Aberto** contém apenas o nó de origem.

A cada passo, o A* retira um nó de **Aberto** para ser analisado. O algoritmo coloca esse nó no final de **Fechado**, verifica se ele é o destino e, se não for, gera os sucessores, que correspondem aos nós que estão conectados ao nó que está sendo analisado. Os sucessores válidos são colocados em **Aberto**. O algoritmo prossegue até que o destino seja alcançado.

PLANEJADOR DE CAMINHOS

Neste planejador de caminhos, cada ponto está conectado a um número variável de outros pontos, que são aqueles para os quais existe uma rota entre eles. Os pontos conectados são os únicos diretamente atingíveis a partir do ponto anterior.

Cada nó do grafo está associado a um ponto do mapa e ao caminho que levou da origem até ele, contendo as seguintes informações:

- A identificação do ponto associado ao nó.
- A identificação da rota que trouxe do nó anterior até ele.
- O comprimento do caminho que passa pelo nó, detalhado em custo passado e futuro, conforme explicado posteriormente.

Não é necessário, nem é permitido, acrescentar nenhum dado ou informação adicional aos nós de **Aberto** ou **Fechado**.

Cada nó n_k tem um custo associado, que é o comprimento do caminho da origem até o nó. Esse custo é denominado de custo passado (**g**). Ele é igual ao custo passado do seu nó antecesor.

o custo n_{k-1} adicionado ao custo da movimentação do antecessor até ele, que é igual ao comprimento da rota que os interliga:

$$g(n_k) = g(n_{k-1}) + \text{comprimento_rota}(n_{k-1}, n_k)$$

O nó inicial (a origem do caminho) tem custo passado nulo ($g = 0$). O comprimento total do caminho será o custo do seu último nó, ou seja, o custo passado (g) do nó destino.

INSERÇÃO DE NOVOS NÓS

Para garantir que o caminho mais curto seja encontrado, o nó retirado de **Aberto** deve ser sempre o de menor custo. Por isso, os nós em **Aberto** devem ser mantidos ordenados em ordem crescente de custo e retira-se sempre o primeiro deles.

Não se deve percorrer o contêiner inteiro para buscar o nó de menor custo a ser retirado.

Como o conjunto estava ordenado no passo anterior, a cada inserção procura-se o local do novo nó no conjunto, que é:

- **antes** do primeiro nó de custo **maior** que ele, caso exista; ou
- ao final do contêiner, caso o novo nó tenha custo maior ou igual que todos os atuais.

Não se deve reordenar o conjunto inteiro a cada vez que um novo nó é inserido.

Cada ponto do mapa só pode ser representado por um único nó em **Aberto** ou em **Fechado**. Quando um sucessor é gerado, verifica-se se um nó que representa o mesmo ponto (ou seja, um nó que tem a mesma id) já não existe em um dos conjuntos:

- Caso o sucessor seja igual (mesma id) que um nó em **Fechado**, ignora-se o sucessor. Nós incluídos em **Fechado** nunca são removidos.
- Caso o sucessor seja igual (mesma id) que um nó em **Aberto**:
 - Caso o sucessor tenha custo igual ou maior que o nó existente, ignora-se o sucessor.
 - Caso o sucessor tenha custo menor que o nó existente, exclui-se o nó existente de **Aberto** e coloca-se o sucessor na posição apropriada em **Aberto**.

ESTIMATIVA DO CUSTO FUTURO

Ao ordenar os nós apenas pelo custo passado, está sendo utilizado o algoritmo de Dijkstra. Isso garante que o caminho mais curto será encontrado primeiro, mas o algoritmo pode ser lento para encontrar o caminho ótimo.

Para acelerar a busca, o algoritmo A* ordena os nós pelo custo total (f), que é a soma do custo passado (g) com o custo futuro (h):

$$f(n_k) = g(n_k) + h(n_k, \text{destino})$$

O custo futuro (h) é baseado em uma estimativa aproximada (heurística) do custo adicional para ir diretamente do nó que está sendo analisado até o nó destino. Garante-se que o caminho mais curto será encontrado se for utilizada uma *heurística otimista*, com valor sempre menor ou igual do que o custo real até o destino. Caso não se use heurística ($h = 0$), o A* recai no algoritmo de Dijkstra.

Neste projeto, será utilizada como heurística a distância euclidiana entre os dois pontos, pois o comprimento da reta entre dois pontos será sempre menor ou igual do que o comprimento do caminho efetivo que precisará ser percorrido usando as rotas existentes. O cálculo da distância euclidiana entre os pontos pode ser feito a partir da latitude e da longitude dos pontos, que são conhecidas.

A distância d entre dois pontos, dadas suas latitudes (φ_1, φ_2) e longitudes (λ_1, λ_2), pode ser calculada de forma aproximada, considerando-se que a Terra é uma esfera, pela fórmula de haversine:

$$\begin{aligned} \Delta\varphi &= \varphi_2 - \varphi_1 & \Delta\lambda &= \lambda_2 - \lambda_1 \\ a &= \sin^2\left(\frac{\Delta\varphi}{2}\right) + \cos\varphi_1 \cos\varphi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right) \\ d &= 2R \sin^{-1}\sqrt{a} = 2R \tan^{-1}\left(\frac{\sqrt{a}}{\sqrt{1-a}}\right) \end{aligned}$$

onde R é o raio da Terra (6371 km) e os ângulos de latitude e longitude devem ser expressos em radianos para que o cálculo computacional seja correto.

LEITURA DO MAPA

As informações do mapa são lidas de arquivos pela função `ler` da classe `Planejador`. Há um arquivo com os pontos, outro arquivo com as rotas. Caso seja detectado algum erro nos arquivos, os dados atuais não devem ser alterados. A função de leitura deve lidar com os erros ou condições excepcionais utilizando o tratamento de exceções em C++.

Os arquivos têm o seguinte formato:

- O início é um cabeçalho pré-definido, com uma quebra de linha ao final.
 - Antes do cabeçalho ou após o cabeçalho e antes da quebra de linha, pode haver separadores (espaços, ENTER ou TAB), que devem ser descartados^{1,2}.
- Após a primeira linha, devem vir os dados (pontos ou rotas). Cada linha pode conter parte de um dado, a totalidade de um dado ou mais de um dado na mesma linha.
- Cada dado tem vários campos, separados por ';' (ponto e vírgula). Não há ';' após o último campo de um dado: há um ou mais separadores antes do próximo dado.
- Antes e após cada campo, pode haver separadores, que devem ser descartados^{1,2}.
- A leitura deve gerar erro caso nenhum dado seja lido de um arquivo.

O arquivo de pontos tem o seguinte formato:

- O cabeçalho deve ser **exatamente**:
`ID;Nome;Latitude;Longitude`
- São 4 campos:
 - 1º campo é a ID do ponto (string).
 - 2º campo é o nome do ponto (string).
 - 3º e 4º campos são a latitude e a longitude do ponto (números).

O arquivo de rotas tem o seguinte formato:

- O cabeçalho deve ser **exatamente**³:
`ID;Nome;Extremidade
1;Extremidade 2;Comprimento`

¹ Lembre-se que o `operator>>` já descarta separadores, seja antes de ler um dado, seja usando a operação específica para isso: `____ >> ws`

² A função `trim` do arquivo `planejador.cpp` já elimina separadores do final de uma string.

³ O cabeçalho fica em uma única linha. Aqui, ele aparece dividido em 2 linhas por falta de espaço.

- São 5 campos:
 - 1º campo é a ID da rota (string).
 - 2º campo é o nome da rota (string).
 - 3º e 4º campos são as ID dos pontos ligados à rota (strings).
 - 5º campo é o comprimento da rota (número).

DICA: NÃO complique a implementação da função de leitura dos arquivos:

- Sempre que possível, leia os dados da stream diretamente para as variáveis ou dados membros nos quais serão armazenados, sem passar por variáveis intermediárias.
- Só é necessário utilizar as funcionalidades básicas de leitura a partir de streams associadas a arquivos, tais como `is_open`, `fail`, `eof`, `operator>>`, `getline` e `ws`. Não é necessário, embora não seja proibido, utilizar outras funcionalidades para leitura de streams (`ignore`, `get`, `peek`, etc.).
- O objetivo **não é** avaliar o conhecimento sobre extração de dados de strings. Portanto, **não pode** ser feita a leitura de todo o arquivo ou de uma linha inteira do arquivo para uma string para, em seguida, extrair os diversos dados e campos processando a string. Não é necessário, **nem permitido**, utilizar métodos avançados da classe string, tais como funções de:
 - modificação (`append`, `insert`, `erase`, `replace`, `resize`, `operator+`, etc.)
 - busca (`compare`, `find`, `find_first_of`, `find_last_of`, etc.)
 - conversão (`substr`, `stoi`, `stod`, etc.)
- Não é necessário, **nem permitido**, utilizar outros tipos de streams, tais como `stringstream`.

Antes de prosseguir com o restante do projeto, faça a implementação e validação da função `ler` da classe `Planejador`, tentando ler arquivos com dados corretos, porém com diferentes formatações, e arquivos incorretos por vários motivos: dados faltantes ou inválidos, pontos ou rotas repetidos, etc.

Estão disponíveis no SIGAA vários exemplos de arquivos corretos e incorretos e o programa teste, que tenta ler esses arquivos.

IMPLEMENTAÇÃO

Tendo em vista os conteúdos a serem avaliados pelo projeto, algumas regras devem ser **obrigatoriamente** seguidas:

- As funções, sempre que aplicável, devem lidar com os erros ou condições excepcionais utilizando o tratamento de exceções em C++ (try, catch, throw).
- As classes (IDPonto, IDRota) e funções (main) que já estão completamente declaradas e implementadas **não** devem ser modificadas de nenhuma forma (por alteração, supressão ou acréscimo).
- As classes (Ponto, Rota) parcialmente declaradas devem receber acréscimos, mas as partes já disponibilizadas (na declaração e na implementação) **não** devem ser modificadas ou suprimidas.
- As classe Noh, a ser declarada e implementada, deverá conter os dados necessários à implementação do algoritmo A* conforme detalhado nessa especificação, **sem acréscimo de outros dados nem substituição dos dados especificados por outros.**
- A classe Planejador está completamente declarada e parcialmente implementada. A implementação deve receber acréscimos, mas toda a declaração e as partes já disponibilizadas na implementação **não** devem ser modificadas ou suprimidas.
- Só deve(m) ser utilizado(s) o(s) contêiner(es) sequenciais (vector, deque ou list) e/ou as adaptações desses contêineres (stack ou queue); **Não** devem ser utilizados os contêineres baseadas em *heaps* (priority_queue) ou em árvores binárias de busca (set, multiset, unordered_set, map, multimap, unordered_map, etc.).
- Os conjuntos Aberto e Fechado devem ser implementados utilizando o contêiner STL mais indicado para cada um deles, levando em conta as operações a serem realizadas durante o algoritmo A*, tais como:
 - Precisa ou não percorrer os elementos.
 - Precisa ou não de acesso por índice.
 - Precisa ou não incluir elementos no início, no meio ou no fim.
 - Precisa ou não excluir elementos do início, do meio ou do fim.

- Sempre que possível, os algoritmos genéricos da STL devem ser utilizados e **NÃO** substituídos por um trecho de código similar, implementado pelo próprio programador, utilizando testes (if) e laços de controle (for). Por exemplo, **NÃO** faça:

```
for (...) { // ou while, do-while
    if (...) {
        ... // O que precisa fazer
    }
}
```

quando poderia fazer (para procurar um único elemento):

```
itr = algoritmo_STL(...);
if (itr != __.end()) {
    ... // O que precisa fazer
}
```

ou então (para procurar todos os elementos que satisfazem o critério de busca):

```
itr = __.begin();
while ( itr != __.end() ) {
    itr=algoritmo_STL(itr, __.end(), ...);
    if (itr != __.end()) {
        ... // O que precisa fazer
        ++itr;
    }
}
```

- Caso possa ser empregado mais de um algoritmo STL, deve ser utilizado o que implementa mais diretamente a função desejada e/ou requer menos programação adicional. Por exemplo:
 - **NÃO** utilize um algoritmo mais complexo (find_if, any_of, etc.) com uma função de teste (por exemplo, uma função anônima) quando um algoritmo que não precisa de uma função de teste (find, sort, etc.) resolveria.
 - **NÃO** utilize algoritmo (lower_ ou upper_bound, min_ ou max_element, sort, unique, etc.) com parâmetro adicional (por exemplo, uma função anônima) quando a versão desse algoritmo sem esse parâmetro resolveria.
 - **NÃO** crie função (clássica ou anônima) com a mesma funcionalidade dos operadores tradicionais (operator==, operator<, etc.). Nesse caso, utilize o algoritmo STL que já se baseia nesse operador e sobrecarregue a implementação dele na classe apropriada.

ALGORITMO A*

```
// TIPOS DE DADOS

// Noh: elementos dos contêineres
// Aberto e Fechado
// NÃO PODE conter dados adicionais
Noh:
    IDPonto    id_pt: id do ponto
    IDRota     id_rt: id da rota do
               antecessor até o ponto

    double     g: custo passado
    double     h: custo futuro
    double     f()=g+h: custo total

// DADOS DE ENTRADA

// Pontos e rotas do mapa
contêiner<Ponto> pontos
contêiner<Rota> rotas
// ID dos pontos de origem e
// destino do caminho
IDPonto        id_origem, id_destino

// DADOS DE SAÍDA

// N° final de nós nos contêineres
// Aberto e Fechado
int             NumAberto, NumFechado
// Sequência de Trechos do
// caminho encontrado
Caminho        C
// Comprimento do caminho
// encontrado
double         Compr
```

```
// ALGORITMO A* (calculaCaminho)

// Obtém pontos de origem e destino
pt_origem ← Ponto de id=id_origem
pt_destino ← Ponto de id=id_destino

// atual ← Noh inicial
atual.id_pt ← id_origem
atual.id_rt ← Ø // IDRota()
atual.g ← 0.0
atual.h ← distância(pt_origem,
                    pt_destino)

// Inicializa os conjuntos de Noh's
Aberto ← Ø // Vazio
Fechado ← Ø // Vazio
incluir(atual, Aberto)

// Laço principal do algoritmo
REPITA
|
| // Lê e exclui o 1° Noh (o de
| // menor custo) de Aberto
| atual ← primeiro(Aberto)
| excluir_primeiro(Aberto)
|
| // Inclui "atual" no final
| // de Fechado
| incluir_final(atual, Fechado)
|
| // Expande se não é solução
| SE (atual.id_pt ≠ id_destino)
| |
| | // Gera sucessores de "atual".
| | // Procura todas as rotas (não
| | // apenas a primeira) ligadas
| | // ao Noh "atual". O sucessor é
| | // o Noh da outra extremidade.
| | REPITA
| | |
| | | // Busca "rota_suc", próxima
| | | // Rota conectada a "atual"
| | | rota_suc ← próxima Rota com
| | | uma das extremidades = atual
| | |
| | | // Achou uma Rota?
| | | SE ( EXISTE(rota_suc) )
| | | |
| | | | // Gera Noh sucessor "suc"
| | | | suc.id_pt ← extremidade de
| | | | rota_suc ≠ atual
| | | | suc.id_rt ← rota_suc.id
| | | | suc.g ← atual.g +
| | | | rota_suc.comprimento
| | | | // Ponto do Noh "suc"
| | | | pt_suc ← Ponto de
| | | | id = suc.id_pt
| | | | suc.h ← distância(pt_suc,
| | | | pt_destino)
| | | |
```

```

| | | // Assume "suc" inédito
| | | eh_inedito ← TRUE
| | |
| | | // Noh=suc em Fechado?
| | | oldF ← buscar(suc.id_pt,
| | |                 Fechado)
| | |
| | | // Achou algum Noh?
| | | SE ( EXISTE(oldF) )
| | | |
| | | | // Noh já existe
| | | | eh_inedito ← FALSE
| | | |
| | | CASO CONTRÁRIO
| | | |
| | | | // Noh=suc em Aberto?
| | | | oldA ← buscar(suc.id_pt,
| | | |                 Aberto)
| | | |
| | | | // Achou algum Noh?
| | | | SE ( EXISTE(oldA) )
| | | | |
| | | | | // Menor custo total?
| | | | | SE (suc.f() < oldA.f())
| | | | | |
| | | | | | // Exclui anterior
| | | | | | excluir(oldA, Aberto)
| | | | | |
| | | | | CASO CONTRÁRIO
| | | | | |
| | | | | | // Noh já existe
| | | | | | eh_inedito ← FALSE
| | | | | |
| | | | | FIM SE (suc.f() < ...
| | | | |
| | | | FIM SE ( EXISTE(oldA) )
| | | |
| | | FIM SE ( EXISTE(oldF) )
| | |
| | | // Não existe igual?
| | | SE ( eh_inedito )
| | | |
| | | | // big ← 1º Noh com f()
| | | | // maior que f() de "suc"
| | | | big ← maior_que(suc.f(),
| | | |                 Aberto)
| | | |
| | | | // Insere "suc" em Aberto
| | | | // antes de "big"
| | | | inserir(suc,big,Aberto)
| | | |
| | | FIM SE ( eh_inedito )
| | |
| | | FIM SE ( EXISTE(rota_suc) )
| | |
| | | // Repita enquanto há rotas com
| | | // "atual" em uma extremidade
| | | ENQUANTO ( EXISTE(rota_suc) )
| | |
| | FIM SE (atual.id_pt ≠ ...

| // Repita enquanto não encontrou
| // solução e há nós em Aberto
ENQUANTO (NÃO(VAZIO(Aberto)) E
           (atual.id_pt≠id_destino))

// Calcula nº de nós da busca
NumAberto ← tamanho(Aberto)
NumFechado ← tamanho(Fechado)

// Inicialmente, caminho "C" vazio
C ← ∅

// Encontrou solução ou não?
SE (atual.id_pt ≠ id_destino)
|
| // Não existe solução
| Compr ← -1.0
| // caminho "C" permanece vazio
|
CASO CONTRÁRIO
|
| // Calcula comprimento do caminho
| Compr ← atual.g
|
| // Refaz o caminho, procurando
| // Nohs antecessores em Fechado
ENQUANTO ( VÁLIDO(atual.id_rt) )
| |
| | // Acrescenta Trecho atual no
| | // topo do caminho "C"
| | incluir(Trecho(atual.id_rt,
| |                 atual.id_pt), C)
| |
| | // Recupera o antecessor.
| | //
| | // Obtém "rota_ant", Rota que
| | // levou até "atual".
| | rota_ant ← Rota de
| |                 id = atual.id_rt
| | // Calcula id do antecessor
| | id_pt_ant ← extremidade de
| |                 rota_ant ≠ atual
| |
| | // Procura Noh igual a
| | // "id_pt_ant" em Fechado
| | atual ← buscar(id_pt_ant,
| |                 Fechado)
| |
FIM ENQUANTO ( VÁLIDO(...) )

// Acrescenta Trecho da origem no
// topo do caminho "C"
incluir(Trecho(∅,atual.id_pt), C)

FIM SE (atual.id_pt ≠ id_destino)

```